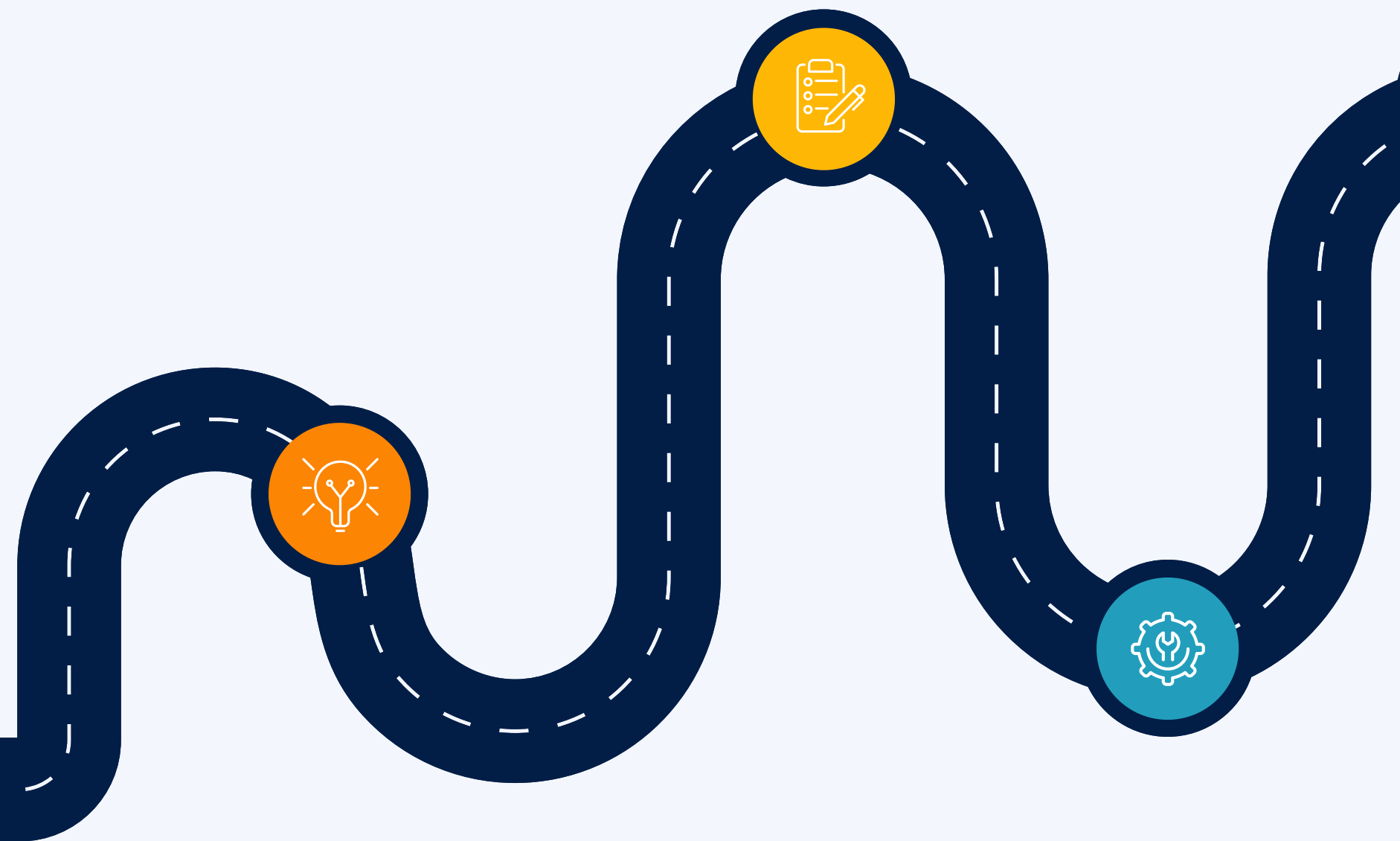


COMPETÊNCIA GERAL

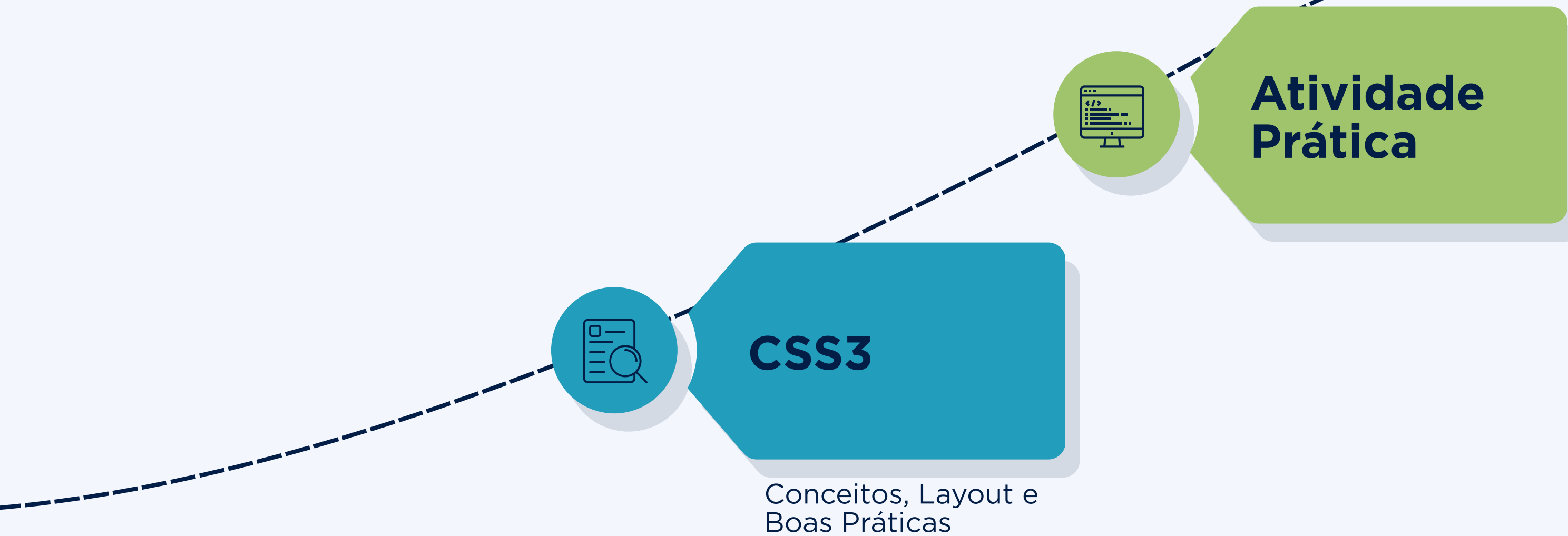
CRIAR SERVIÇOS WEB COM REST

Projeto WEB com REST



CONTEÚDOS

Encontro 02



O QUE É O CSS?

CSS é a linguagem que **descreve** como o **HTML** deve ser **apresentado**.

- **Três ideias mandam aqui:**

Cascata, Especificidade e Herança

- **Cascata**

Na cascata, quando regras conflitam, vence a que tiver maior especificidade ou a última definida.

- **Herança**

A herança passa algumas propriedades, como color e font, dos pais para os filhos.

BOX MODEL (CONTENT, PADDING, BORDER, MARGIN)

*Todo elemento é uma caixa:
conteúdo, padding, borda e
margem.*

- **Entenda as camadas da caixa**

Se não entender isso, os espaçamentos e alinhamentos vão parecer aleatórios.

- **Use box-sizing: border-box**

Um ajuste que quase sempre ajuda é box-sizing: border-box, porque ele faz width incluir padding e borda, evitando contas mentais. Visualize cada camada: o padding é o 'acolchoado', a borda é o contorno, e a margem é o espaço externo entre caixas.

- **Inspecione no DevTools**

Lembrar de sempre conferir o Box Model no DevTools quando algo 'quebrar'.

UNIDADES E TAMANHOS FLUÍDOS

- **Use rem para tipografia e escala**

No CSS moderno, pense em escala fluída. Use rem para fontes e espaçamentos relativos ao tamanho base do documento.

- **%, vh, vw para elementos fluidos**

Use %, vh e vw para larguras e alturas adaptáveis.

- **clamp(min, ideal, max) para limites**

E use clamp(min, ideal, max) para controlar crescimento fluído com limites seguros. Assim garantimos legibilidade no celular e bom aproveitamento no desktop sem dezenas de media queries.

SELETORES QUE ECONOMIZAM CLASSES

- **Atributo, :is(), :not(), :has()**

Selecionar bem reduz HTML poluído. Padrões úteis: seletores de atributo, pseudo-classes como :not() e :is() para simplificar combinações, e o moderno :has() para reagir ao estado dos filhos

- **Pseudo-elementos ::before/::after**

Pseudo-elementos ::before e ::after criam ornamentos sem novos elementos no HTML.

- **clamp(min, ideal, max) para limites**

Lembre: seletores claros ajudam manutenção e evitam cadeias gigantes de especificidade.

DISPLAY: BLOCK, INLINE, INLINE-BLOCK, FLEX E GRID

- **Escolha o display certo para cada caso**

Escolher o display certo evita gambiarra. block ocupa a linha toda; inline flui no texto e ignora width/height; inline-block permite dimensão sem quebrar o fluxo.

- **Flexbox: 1D; Grid: 2D**

Para layout de uma dimensão, use Flexbox; para duas dimensões, use Grid.

- **flow-root para conter floats**

flow-root resolve problemas de 'clearfix' e contenção de floats. Entender display é metade do layout.

CORES E CONTRASTE (ACESSIBILIDADE)

- **Contraste AA no mínimo**

Cor é estética e acessibilidade. Busque contraste AA no mínimo para textos normais.

- **Prefira HSL para variar brilho/saturação**

Prefira HSL quando precisar variar brilho e saturação mantendo a mesma matiz; isso agiliza criar estados hover mais claros ou escuros. Gradientes dão profundidade, mas cuidado para não prejudicar a leitura do texto.

- **Valide com ferramentas do navegador**

Ferramentas de contraste no navegador ajudam a validar rapidamente.

TIPOGRAFIA: LEGIBILIDADE E RITMO

- **Stack de fontes confiável**

Tipografia guia o olho do usuário. Use uma stack de fontes confiável e defina line-height sem unidade para herdar bem.

- **line-height sem unidade**

line-height sem unidade no CSS define um espaçamento entre linhas que é proporcional ao font-size do elemento e seus descendentes, sendo a forma recomendada por sua flexibilidade.

- **Tamanhos fluídos com clamp()**

Para tamanhos que se adaptam, clamp() resolve 80% dos casos. Trate quebra de linha e truncamento com text-wrap e text-overflow quando necessário. Títulos com peso, corpo com conforto, e hierarquia consistente deixam a interface mais profissional.

BORDAS, RADIUS, SOMBRAS E FOCO

- **Detalhes que elevam a UI**

Pequenos detalhes fazem a interface parecer ‘acabada’. border-radius suave, box-shadow discreta, e estado de foco visível com :focus-visible, isso é crucial para teclado e acessibilidade.

- **Use :focus-visible (não remova outline)**

Use outline para foco em vez de removê-lo: quem navega por teclado depende desse feedback.

- **Transições sutis em hover/foco**

Transições curtas em foco e hover comunicam interatividade sem exagero.

FLEXBOX: MODELO MENTAL

- **Eixo principal e eixo cruzado**

Flexbox resolve alinhamento em uma dimensão. Pense em eixo principal e eixo cruzado.

- **justify-content e align-items**

- justify-content alinha no eixo principal;
- align-items no cruzado.

- **gap e flex: grow shrink basis**

- gap substitui aquelas margens cheias de exceções.
- flex: grow shrink basis define como cada item expande, encolhe e parte de um tamanho base.
- Se você entender esses quatro pontos, monta 90% dos headers, toolbars e grids simples.

PADRÕES DE FLEXBOX PRONTOS

- **Centralização perfeita**

Alguns padrões valem decorar. Centralização perfeita com:

- `display:flex;`
- `align-items:center;`
- `justify-content:center.`

- **Grid de cards com wrap**

Grids de cards fluidos com flex-wrap e gap para se adaptar a qualquer largura.

- **Espaçadores com flex:1**

Espaçadores com flex:1 criam áreas elásticas entre itens. A ideia é ter meia dúzia de blocos ‘copiar e colar’ para acelerar qualquer layout.

VARIÁVEIS CSS (CUSTOM PROPERTIES) E TEMAS

- **Tokens no :root**

- Definimos no :root os tokens principais e referenciamos com var(--nome).

- **Tema claro/escuro com prefers-color-scheme**

- Custom Properties vivem no próprio CSS e mudam em tempo real, perfeito para tema claro/escuro ou personalização por usuário.
- Com @media (prefers-color-scheme: dark), ajustamos automaticamente o tema em navegadores modernos. Isso traz consistência visual e menos repetição de valores.

RESPONSIVIDADE PRÁTICA (MOBILE-FIRST)

- **Comece do menor para o maior**

Comece do menor para o maior: estilos básicos funcionam no mobile sem media queries. Depois, adicione quebras com @media (min-width: ...) para ampliar o layout.

- **Poucos breakpoints previsíveis**

Tenha 3-4 breakpoints previsíveis e use clamp() e unidades fluídas para reduzir a quantidade de queries.

- **Reduza media queries com unidades fluídas**

O foco é manter o conteúdo legível e a navegação confortável em qualquer largura.

INTERAÇÕES: TRANSIÇÕES E MICROANIMAÇÕES

- **Prefira transform e opacity**

Priorize transform e opacity para performance suave. Uma elevação sutil no hover de um card ou um feedback ao focar um input já melhoram a percepção de qualidade.

- **Animação é tempero, não o prato principal**

- **Feedback claro em hover/foco**

Evite transições longas e exageradas. O usuário deve sentir que a interface responde, não que está esperando por efeitos.

ORGANIZAÇÃO E QUALIDADE

- **Base, componentes, layouts e utilitários**

CSS escala quando a organização é clara: separe base, componentes, layouts e utilitários.

- **BEM para clareza**

Use uma convenção simples, como BEM, para comunicar a função das classes.

- **Revise contraste, foco e CSS morto**

Revise contraste, estados de foco e CSS não utilizado. Ative font-display: swap nas fontes para melhorar carregamento. E sempre valide no DevTools o que a cascata está realmente aplicando.

VAMOS PARA A PRÁTICA!

Link:

<https://github.com/sheilapaiva/atividade-pratica-css>

- **Opção A (rápida): clique no botão verde “Code” → “Download ZIP”**
- **Opção B (git): `git clone https://github.com/sheilapaiva/atividade-pratica-css`**